

Material complementario

Semana 4

Introducción

En un mercado competitivo, comprender la relación entre precios y demanda resulta muy importante para la toma de decisiones estratégicas.

En este material, exploraremos el análisis de sensibilidad de precios mediante técnicas de modelado predictivo. Aprenderás a recolectar y preparar datos, seleccionar las variables más relevantes, entrenar modelos de predicción y evaluar su rendimiento. Además, aplicaremos el concepto de **elasticidad precio de la demanda**, una herramienta fundamental para entender cómo los cambios en los precios afectan el comportamiento del consumidor.

Al finalizar, serás capaz de aplicar estos conocimientos para interpretar datos del mercado y formular estrategias de precios basadas en evidencia.

Análisis del modelado de sensibilidad de precios

El análisis de sensibilidad de precios es una técnica utilizada para evaluar cómo los cambios en los precios afectan la demanda y el comportamiento del consumidor. Mediante el modelado de sensibilidad de precios, las empresas pueden determinar estrategias óptimas de fijación de precios y comprender la elasticidad de sus productos o servicios.

La literatura sobre el tema destaca enfoques cuantitativos como la regresión lineal y no lineal, los modelos de series temporales y las técnicas de *machine learning*, que permiten identificar patrones de sensibilidad al precio en distintos segmentos de mercado. También se ha estudiado el impacto de factores adicionales, como la competencia, la percepción del valor y las promociones, que influyen en la respuesta del consumidor ante variaciones en el precio.

Un aspecto destacable en estos estudios es la importancia de la calidad de los datos y la selección de variables relevantes, ya que una mala elección de características puede distorsionar los resultados del análisis de sensibilidad. Es fundamental combinar técnicas estadísticas con el conocimiento experto del mercado para garantizar que los resultados obtenidos sean representativos y accionables en distintos escenarios comerciales.

Selección de variables

Para desarrollar un modelo de análisis de sensibilidad de precios efectivo, es vital seleccionar adecuadamente las variables que influyen la demanda y la disposición a pagar de los consumidores.

Criterio experto

El criterio experto implica la selección de variables basándose en el conocimiento del mercado y del negocio. En el caso de la sensibilidad de precios, los analistas pueden identificar factores clave como:

- Precio actual del producto
- Precio de productos sustitutos y complementarios
- Descuentos y promociones
- Historial de compras de los clientes
- Segmento de mercado y preferencias

Aplicación: antes de aplicar técnicas estadísticas, los expertos pueden realizar un análisis exploratorio de los datos y descartar variables irrelevantes o redundantes.

Multicolinealidad en variables predictoras

La multicolinealidad ocurre cuando dos o más variables predictoras están altamente correlacionadas, lo que puede generar inestabilidad en los modelos de regresión y afectar la interpretación de los coeficientes.

Detección de multicolinealidad:

Matriz de correlación

```
import seaborn as sns
import matplotlib.pyplot as plt

correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Matriz de correlación')
plt.show()
```

Factor de Inflación de la Varianza (VIF)

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd
```

```
X = df[['Precio', 'Descuento', 'Demanda', 'Competencia']]
vif_data = pd.DataFrame()
vif_data['Variable'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]
print(vif_data)
```

Un VIF superior a 10 indica una alta multicolinealidad.

Soluciones para la multicolinealidad:

1. Eliminar una de las variables correlacionadas si aportan información redundante.
2. Transformaciones de datos, como el uso de PCA (Análisis de Componentes Principales).
3. Uso de modelos de regresión regularizados, como Ridge o Lasso.

Resultado esperado: un conjunto de variables seleccionadas de manera óptima, eliminando redundancias y mejorando la estabilidad del modelo de análisis de sensibilidad de precios.

¿Cómo seleccionar y gestionar variables para el análisis de sensibilidad de precios?



Figura 1. Métodos para seleccionar y gestionar variables en el análisis de sensibilidad de precios

Recolección de precios

El análisis de sensibilidad de precios requiere datos precisos y representativos que reflejen el comportamiento real del mercado. La recolección de precios es un proceso clave que permite alimentar los modelos con información relevante proveniente de diversas fuentes.

Fuentes de datos

Para realizar un análisis efectivo, es fundamental obtener datos de diferentes fuentes, asegurando una visión integral del mercado y el consumidor.

- Ventas internas: datos históricos de transacciones, precios aplicados, descuentos y respuesta del consumidor ante variaciones de precio.
- Datos de mercado: información sobre la competencia, estrategias de *pricing*, variaciones de demanda y factores externos que influyen en la fijación de precios.
- Datos demográficos: características de los consumidores, como nivel de ingresos, ubicación geográfica y patrones de compra, que pueden afectar la sensibilidad al precio.

Ejemplo de integración de datos: las empresas pueden combinar sus registros internos con datos externos de mercado y demográficos para obtener modelos más robustos.

Extracción: herramientas y API

La recolección de precios puede realizarse mediante distintas herramientas y API que permiten automatizar la obtención de datos en tiempo real.

- API de datos de mercado: servicios como Google Trends, Open Data y plataformas de análisis financiero proporcionan datos sobre tendencias y precios en distintos sectores.
- *Scraping web*: herramientas como BeautifulSoup y Scrapy permiten extraer información de precios desde sitios web de competidores.
- Bases de datos internas: sistemas CRM y ERP almacenan datos transaccionales y de clientes que pueden ser utilizados para modelar la sensibilidad de precios.

Ejemplo de extracción con Python:

```
import requests

url = "https://api.ejemplo.com/precios"
response = requests.get(url)
data = response.json()
print(data)
```

Limpieza de datos

Una vez recolectados los datos, es necesario limpiarlos y prepararlos para su uso en modelos de análisis de precios. La calidad de los datos es un factor determinante en la precisión de los resultados.

Manejo de valores faltantes

Los valores nulos pueden sesgar el análisis y afectar la confiabilidad de los modelos. Se pueden manejar mediante:

- Imputación con medidas centrales (media, mediana o moda).

- Eliminación de registros incompletos, si la cantidad de datos faltantes es considerablemente baja.

Ejemplo en Python:

```
df.fillna(df.mean(), inplace=True) # Imputa valores faltantes con la media
```

Eliminación de duplicados

La presencia de registros duplicados puede distorsionar los resultados. Para evitarlo, se deben identificar y eliminar estos registros.

Ejemplo en Python:

```
df.drop_duplicates(inplace=True)
```

Normalización de datos

La normalización permite que todas las variables estén en una escala comparable, evitando que unas dominen sobre otras en los modelos de predicción.

Ejemplo de normalización Min-Max:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[['precio']] = scaler.fit_transform(df[['precio']])
```

Tratamiento de *outliers*

Los valores atípicos pueden alterar los resultados del análisis de sensibilidad de precios. Se pueden manejar mediante:

- Eliminación de valores extremos si se consideran errores.
- Winsorización, que recorta los valores extremos sin eliminarlos.

Ejemplo de detección de *outliers*:

```
import seaborn as sns  
  
sns.boxplot(x=df['precio'])
```

Entrenamiento del modelo

El entrenamiento de modelos predictivos es una etapa clave en el análisis de sensibilidad de precios. A través de distintas técnicas de *machine learning*, es posible estimar cómo los cambios en el precio afectan la demanda y tomar decisiones estratégicas basadas en datos. A continuación, se presentan los principales algoritmos utilizados en este contexto.

Regresión lineal

La regresión lineal es el modelo más simple y ampliamente utilizado en la predicción de sensibilidad de precios. Se basa en la relación lineal entre una variable dependiente (como la demanda) y una o más variables independientes (como el precio, la competencia y las promociones).

Ecuación matemática:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$$

Donde:

- Y es la variable objetivo (demanda).
- X_n son las variables predictoras.

- β_n son los coeficientes del modelo.
- ε es el error.

Implementación en Python:

```
from sklearn.linear_model import LinearRegression

modelo_lr = LinearRegression()
modelo_lr.fit(X_train, y_train)
y_pred = modelo_lr.predict(X_test)
```

Ventajas:

- Fácil de interpretar.
- Rápido de entrenar.

Limitaciones:

- No captura relaciones no lineales.
- Sensible a la multicolinealidad.

Árbol de decisión

Los árboles de decisión son modelos que dividen los datos en ramas basadas en reglas de decisión derivadas de las características predictoras. Son útiles cuando hay relaciones no lineales entre las variables.

Implementación en Python:

```
from sklearn.tree import DecisionTreeRegressor

modelo_dt = DecisionTreeRegressor(max_depth=5)
modelo_dt.fit(X_train, y_train)
y_pred = modelo_dt.predict(X_test)
```

Ventajas:

- Capta relaciones no lineales.
- Fácil de interpretar cuando es poco profundo.

Limitaciones:

- Puede sobreajustarse a los datos de entrenamiento.
- Sensible a datos ruidosos.

Random Forest

El Random Forest es un conjunto de múltiples árboles de decisión que combinan sus predicciones para mejorar la precisión y reducir el sobreajuste. Se basa en la técnica de *bagging*, que entrena cada árbol con diferentes subconjuntos de los datos.

Implementación en Python:

```
from sklearn.ensemble import RandomForestRegressor

modelo_rf = RandomForestRegressor(n_estimators=100, max_depth=10,
random_state=42)
modelo_rf.fit(X_train, y_train)
y_pred = modelo_rf.predict(X_test)
```

Ventajas:

- Reduce el sobreajuste en comparación con un solo árbol.
- Maneja grandes volúmenes de datos y relaciones complejas.

Limitaciones:

- Más lento en entrenamiento y predicción.

- Menos interpretable que un árbol simple.

XGBoost

XGBoost (Extreme Gradient Boosting) es una técnica basada en árboles que optimiza el aprendizaje mediante un proceso secuencial en el que cada nuevo árbol corrige los errores del anterior. Es ampliamente utilizado por su alta precisión en problemas de predicción.

Implementación en Python:

```
import xgboost as xgb

modelo_xgb = xgb.XGBRegressor(n_estimators=100, learning_rate=0.1,
max_depth=5)
modelo_xgb.fit(X_train, y_train)
y_pred = modelo_xgb.predict(X_test)
```

Ventajas:

- Excelente precisión y rendimiento en grandes volúmenes de datos.
- Control eficiente del sobreajuste mediante regularización.

Limitaciones:

- Mayor tiempo de entrenamiento.
- Requiere ajuste fino de hiperparámetros.

Red neuronal

Las redes neuronales son modelos avanzados inspirados en el funcionamiento del cerebro humano. Se componen de múltiples capas de neuronas que aprenden patrones complejos en los datos.

Implementación en Python con TensorFlow/Keras:

```
import tensorflow as tf
from tensorflow import keras

modelo_nn = keras.Sequential([
    keras.layers.Dense(64, activation='relu',
input_shape=(X_train.shape[1],)),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1)
])

modelo_nn.compile(optimizer='adam', loss='mse')
modelo_nn.fit(X_train, y_train, epochs=50, batch_size=16, verbose=1)
y_pred = modelo_nn.predict(X_test)
```

Ventajas:

- Capta relaciones extremadamente complejas y no lineales.
- Escalable para grandes volúmenes de datos.

Limitaciones:

- Difícil de interpretar.
- Alto costo computacional.

Comparación de modelos:

Modelo	Precisión	Interpretabilidad	Tiempo de entrenamiento	Manejo de no linealidad
Regresión lineal	Media	Alta	Bajo	No
Árbol de decisión	Media	Media	Medio	Sí
Random Forest	Alta	Baja	Medio	Sí
XGBoost	Muy alta	Baja	Alto	Sí
Red Neuronal	Muy alta	Muy baja	Muy alto	Sí

Tabla 1. Comparación de distintos modelos utilizados en el análisis de sensibilidad de precios, evaluando su precisión, interpretabilidad, tiempo de entrenamiento y capacidad para manejar relaciones no lineales

Técnicas de evaluación y validación

Para determinar la precisión y confiabilidad de un modelo en el análisis de sensibilidad de precios, es fundamental aplicar técnicas de evaluación y validación. Estas permiten medir el rendimiento del modelo, identificar posibles errores y garantizar que las predicciones sean aplicables en escenarios reales.

Validación cruzada (*cross-validation*)

La validación cruzada es un método que divide los datos en múltiples subconjuntos (o *folds*), donde el modelo se entrena en una parte de los datos y se prueba en otra. Esto evita que el modelo dependa demasiado de un solo conjunto de entrenamiento y mejora su capacidad de generalización.

Método k-fold cross-validation

- Se divide el dataset en k partes.
- Se entrena el modelo en k-1 partes y se valida en la restante.
- Se repite el proceso k veces, cambiando la parte usada para la validación.

Implementación en Python:

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
scores = cross_val_score(modelo, X, y, cv=5,
                          scoring='neg_mean_squared_error')
rmse_scores = (-scores) ** 0.5
print(f"RMSE promedio: {rmse_scores.mean()}")
```

Ventajas:

- Reduce el riesgo de sobreajuste.
- Evalúa el modelo con diferentes particiones de datos.

Limitaciones:

- Aumenta el tiempo de entrenamiento, ya que se ejecuta varias veces.

RMSE (Root Mean Squared Error)

El RMSE mide la magnitud del error en las predicciones del modelo. Cuanto menor sea el RMSE, mejor será la precisión del modelo.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Implementación en Python:

```
from sklearn.metrics import mean_squared_error
import numpy as np

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"RMSE: {rmse}")
```

Interpretación:

- RMSE alto indica un modelo con errores significativos.
- RMSE bajo sugiere que el modelo realiza buenas predicciones.

MAE (*Mean absolute error*)

El MAE mide el error medio absoluto entre las predicciones y los valores reales.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Implementación en Python:

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred)
print(f"MAE: {mae}")
```

Diferencia entre RMSE y MAE:

- MAE es menos sensible a valores extremos que RMSE.
- RMSE penaliza más los errores grandes debido a la elevación al cuadrado.

R-squared

El R^2 mide qué porcentaje de la variabilidad de la variable dependiente es explicado por el modelo.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Implementación en Python:

```
from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)
print(f"R²: {r2}")
```

Interpretación:

- Un R^2 cercano a 1 indica un buen ajuste del modelo.
- Un R^2 cercano a 0 significa que el modelo no explica la variabilidad de los datos.

Gráfico de residuales y análisis de errores

El gráfico de residuales es una herramienta visual que muestra la diferencia entre las predicciones del modelo y los valores reales. La interpretación de un gráfico de residuales se puede realizar de la siguiente manera:

- Distribución aleatoria alrededor de cero: indica que el modelo es adecuado.
- Patrones visibles: sugieren que el modelo no ha capturado completamente la relación entre las variables.

Implementación en Python:

```
import matplotlib.pyplot as plt

residuales = y_test - y_pred
plt.scatter(y_pred, residuales)
plt.axhline(y=0, color='red', linestyle='--')
plt.title('Gráfico de Residuales')
plt.xlabel('Predicciones')
plt.ylabel('Errores (Residuales)')
plt.show()
```

Análisis de errores:

- Si los errores son grandes en ciertos rangos de datos, el modelo podría no ser adecuado para todas las situaciones.
- Una distribución no homogénea puede indicar la necesidad de transformar los datos o usar otro tipo de modelo.

Mejoramiento del modelo

Después de evaluar el rendimiento del modelo con métricas como RMSE, MAE y R^2 , el siguiente paso es optimizarlo para mejorar su precisión y generalización. Uno de los enfoques más efectivos para lograrlo es el ajuste de hiperparámetros, que permite encontrar la mejor configuración de un modelo sin modificar los datos de entrada.

Ajuste de hiperparámetros

Los hiperparámetros son configuraciones del modelo que no se aprenden directamente de los datos, sino que deben ser definidos antes del entrenamiento. Su ajuste puede mejorar significativamente la capacidad predictiva del modelo.

Ejemplo de hiperparámetros en distintos modelos:

- Regresión Ridge y Lasso: valor de penalización α .
- Árboles de decisión: profundidad máxima, criterio de división.
- Random Forest: número de árboles, profundidad máxima, número mínimo de muestras por división.
- XGBoost: tasa de aprendizaje, número de iteraciones, regularización λ .
- Red neuronal: número de capas ocultas, neuronas por capa, función de activación.

Grid Search y Random Search

Estas son dos estrategias comunes para el ajuste de hiperparámetros:

- Grid Search: explora todas las combinaciones posibles de hiperparámetros.
- Random Search: selecciona aleatoriamente combinaciones de hiperparámetros y evalúa su rendimiento.

Ejemplo en Python con Grid Search para Random Forest:

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor

parametros = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 20],
    'min_samples_split': [2, 5, 10]
}

modelo_rf = RandomForestRegressor()
```

```
grid_search = GridSearchCV(modelo_rf, parametros, cv=5,  
scoring='neg_mean_squared_error')  
grid_search.fit(X_train, y_train)  
  
print(f"Mejores hiperparámetros: {grid_search.best_params_}")  
print(f"Mejor RMSE: {(-grid_search.best_score_) ** 0.5}")
```

Ventajas del ajuste de hiperparámetros:

- Mejora la precisión del modelo.
- Reduce el sobreajuste o subajuste.
- Optimiza el rendimiento en distintos conjuntos de datos.

Desafíos:

- Puede ser computacionalmente costoso.
- Requiere tiempo para encontrar la mejor combinación.

¿Qué método de búsqueda de hiperparámetros debería usarse?

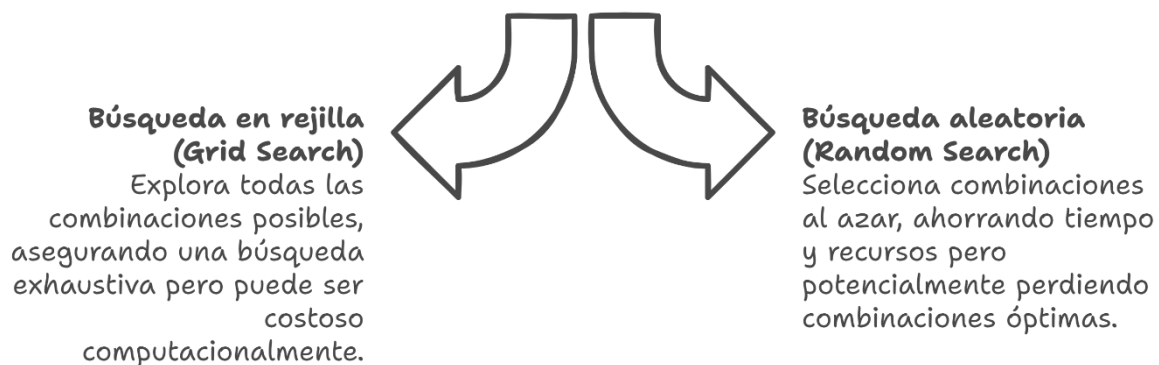


Figura 2. Comparación entre Grid Search y Random Search en la búsqueda de hiperparámetros

Cálculo de elasticidad de precio

La elasticidad precio de la demanda (EPD) mide la sensibilidad de la demanda de un producto ante cambios en su precio. Se calcula con la siguiente fórmula:

$$E_p = \frac{\% \Delta Q}{\% \Delta P}$$

Donde:

- E_p = Elasticidad precio de la demanda.
- $\% \Delta Q$ = Cambio porcentual en la cantidad demandada.
- $\% \Delta P$ = Cambio porcentual en el precio.

Fórmula desglosada:

$$E_p = \frac{\left(\frac{Q_2 - Q_1}{Q_1} \right) \times 100}{\left(\frac{P_2 - P_1}{P_1} \right) \times 100}$$

Donde:

- Q_1 y Q_2 son las cantidades demandadas antes y después del cambio de precio.
- P_1 y P_2 son los precios antes y después del cambio.

Clasificación de la elasticidad

- $E_p > 1 \rightarrow$ Demanda elástica: la demanda responde más que proporcionalmente al cambio en el precio.
- $E_p < 1 \rightarrow$ Demanda inelástica: la demanda es poco sensible al cambio de precio.

- $E_p = 1 \rightarrow$ Elasticidad unitaria: la variación en precio genera un cambio proporcional en la demanda.

Ejemplo de cálculo en Python:

```
def calcular_elasticidad(q1, q2, p1, p2):  
    cambio_q = ((q2 - q1) / q1) * 100  
    cambio_p = ((p2 - p1) / p1) * 100  
    elasticidad = cambio_q / cambio_p  
    return elasticidad  
  
# Ejemplo de datos  
q1, q2 = 1000, 800  
p1, p2 = 50, 55  
  
ep = calcular_elasticidad(q1, q2, p1, p2)  
print(f"Elasticidad precio de la demanda: {ep}")
```

Si $E_p = -1.5$, la demanda es elástica (cambia con el precio). Si $E_p = -0.5$, la demanda es inelástica (no varía mucho ante cambios de precio).

Análisis e interpretación

El análisis de la elasticidad de precios permite evaluar cómo responden los consumidores ante variaciones en los precios y determinar estrategias de fijación de precios basadas en evidencia. Este proceso incluye la clasificación de la elasticidad, la evaluación del impacto de los cambios de precio, la visualización de datos y la formulación de recomendaciones estratégicas.

Clasificación de la elasticidad

Como lo pudiste ver anteriormente, la elasticidad precio de la demanda (E_p) se clasifica en tres categorías principales:

- Demanda elástica $E_p > 1$: un pequeño cambio en el precio genera un cambio mayor en la cantidad demandada. Los consumidores son sensibles al precio.
- Demanda inelástica $E_p < 1$: la cantidad demandada varía en menor proporción que el cambio en el precio. Los consumidores no reaccionan fuertemente a cambios en precios.
- Elasticidad unitaria $E_p = 1$: la variación en la cantidad demandada es proporcional al cambio en el precio.

Evaluación del impacto de cambios en el precio

El cálculo de la elasticidad permite simular distintos escenarios y evaluar cómo afectarán los cambios de precio a la demanda.

Ejemplo de simulación en Python:

```
def impacto_cambioPrecio(q1, p1, ep, cambioPrecio):  
    p2 = p1 * (1 + cambioPrecio / 100)  
    q2 = q1 * (1 + (ep * cambioPrecio / 100))  
    return p2, q2  
  
# Datos de ejemplo  
q1, p1 = 1000, 50 # Cantidad inicial y precio inicial  
ep = -1.5 # Elasticidad estimada  
cambioPrecio = -10 # Reducción del 10% en el precio  
  
p2, q2 = impacto_cambioPrecio(q1, p1, ep, cambioPrecio)
```

```
print(f"Nuevo precio: {p2}, Nueva demanda: {q2}")
```

Interpretación: si el modelo predice que una reducción del 10% en el precio incrementará la demanda en un 15%, la empresa puede considerar una estrategia de reducción de precios para aumentar sus ingresos.

Visualización de la relación precio-demanda

Los gráficos permiten analizar la relación entre los precios y la cantidad demandada, facilitando la toma de decisiones basada en tendencias.

Ejemplo de gráfico en Python:

```
import matplotlib.pyplot as plt
import numpy as np

# Simulación de datos
precios = np.linspace(10, 100, 10)
demanda = 5000 / precios # Relación inversa entre precio y demanda

plt.plot(precios, demanda, marker='o', linestyle='-', color='b')
plt.title("Relación Precio-Demanda")
plt.xlabel("Precio")
plt.ylabel("Cantidad Demandada")
plt.grid()
plt.show()
```

Análisis gráfico:

- Una curva empinada sugiere demanda elástica.
- Una curva más plana indica demanda inelástica.

Recomendaciones estratégicas

Basándose en la elasticidad y los análisis gráficos, se pueden formular estrategias de precios para maximizar los ingresos:

- Si la demanda es elástica ($E_p > 1$), una reducción de precio puede aumentar significativamente las ventas y generar mayores ingresos totales.
- Si la demanda es inelástica ($E_p < 1$), es posible aumentar los precios sin afectar drásticamente la demanda, optimizando los márgenes de ganancia.
- Para productos con elasticidad unitaria ($E_p = 1$), los cambios en precios deben ser cuidadosamente evaluados, ya que los ingresos totales permanecen constantes.

Retroalimentación y ajuste del modelo

El análisis de sensibilidad de precios es un proceso dinámico que requiere ajustes continuos. La retroalimentación basada en nuevos datos permite mejorar la precisión del modelo y adaptarlo a cambios en el mercado.

Estrategias para la retroalimentación del modelo:

- Actualización de datos: incorporar información reciente sobre ventas y precios.
- Reentrenamiento del modelo: ajustar hiperparámetros o probar nuevos enfoques de modelado.
- Segmentación del mercado: analizar elasticidad en diferentes segmentos de clientes.

- Monitoreo de tendencias: evaluar cambios en la sensibilidad al precio a lo largo del tiempo.